Hands-On Machine Learning

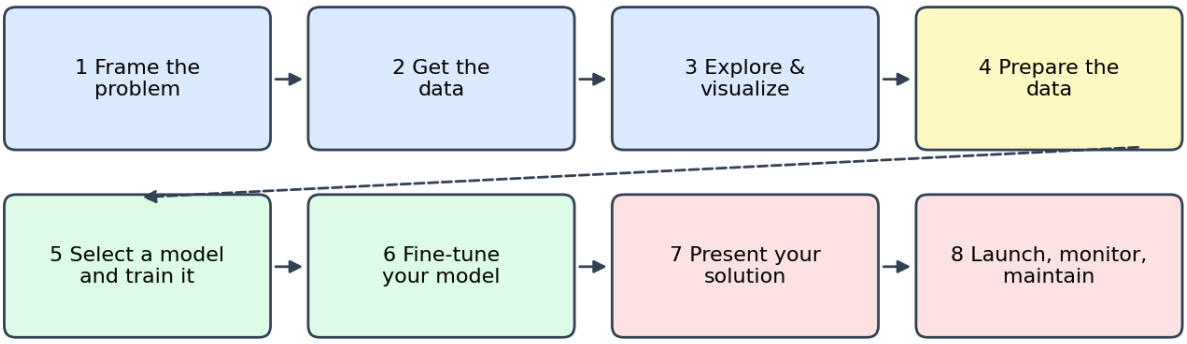
Chapter 2 — End-to-End Machine Learning Project

California Housing Prices

Summary of your notebook + the missing points filled in + figures and code for revision

From your notebook **Added to fill gaps** **Note / correction**

All numbers in the green **Output (real run)** boxes came from actually running the code on the `housing.csv` file (20,640 districts). Numbers marked "from the book" are the 3rd-edition numbers: the copy of the data we loaded gives close, but not 100% identical, numbers. The 8 steps of any ML project: this chapter walks through all of them in order.



1 The big picture and the steps of an ML project

From your notebook

This chapter works with you on a project from start to end, and the goal is to show you the main steps of any ML project. The data we work on is **California Housing Prices**; it is not modern, but it has many educational features, so the book considers it educational data, and added a categorical column to it (`ocean_proximity`) and removed some features.

I am not going to write every piece of code in the book; I will write the code, terms and things I see for the first time.

Addition

Frame the problem (in this project):

- `Supervised learning` : we have labels: `median_house_value` (the real price for each district).
- `Regression` : we predict a number, not a class.
- `Multiple regression` : we use more than one feature to predict. And `Univariate` : we predict one value for each district.
- `Batch learning` : there is no data flowing in continuously, and the data is loaded fully into memory.
- The data has **20,640** districts in this file.

Steps 7 and 8 (Present / Launch) are not in your notebook, so they are summarized at the end of the file (section 17).

2 A look at the data

Addition

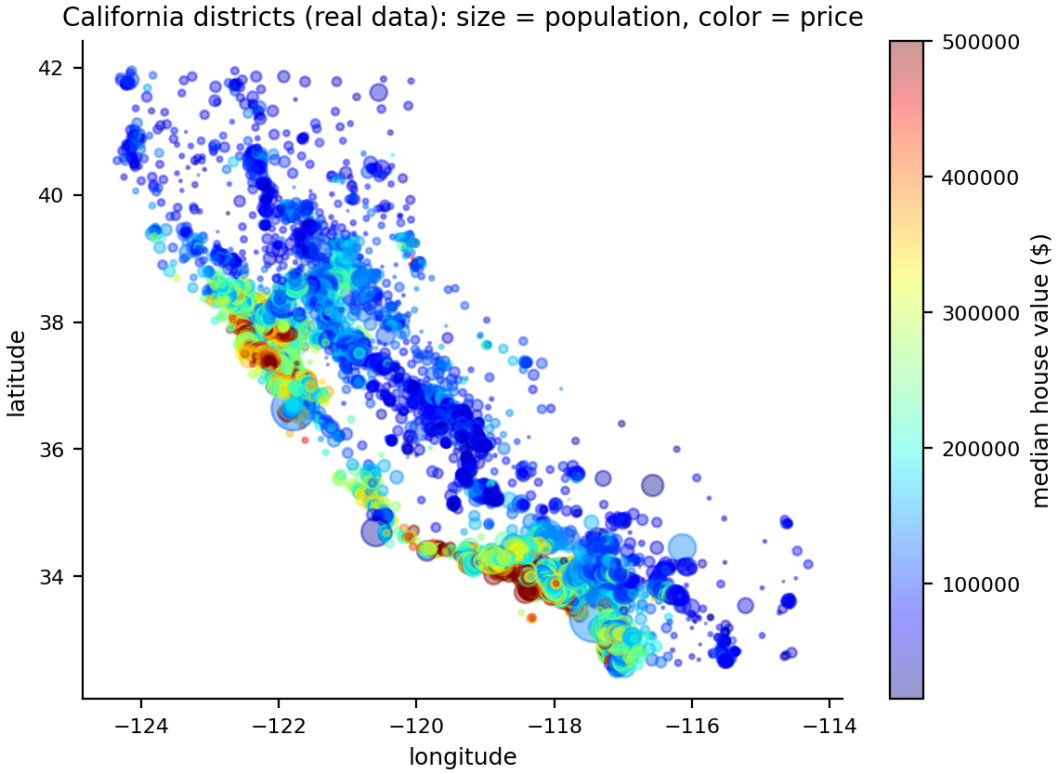
Before anything, we download the data and look at it (steps 2 and 3 of the project):

```
import pandas as pd
housing_full = pd.read_csv("housing.csv")
print(housing_full.shape)
print(housing_full["ocean_proximity"].value_counts())
print(housing_full["total_bedrooms"].isna().sum(), "missing values in total_bedrooms")
```

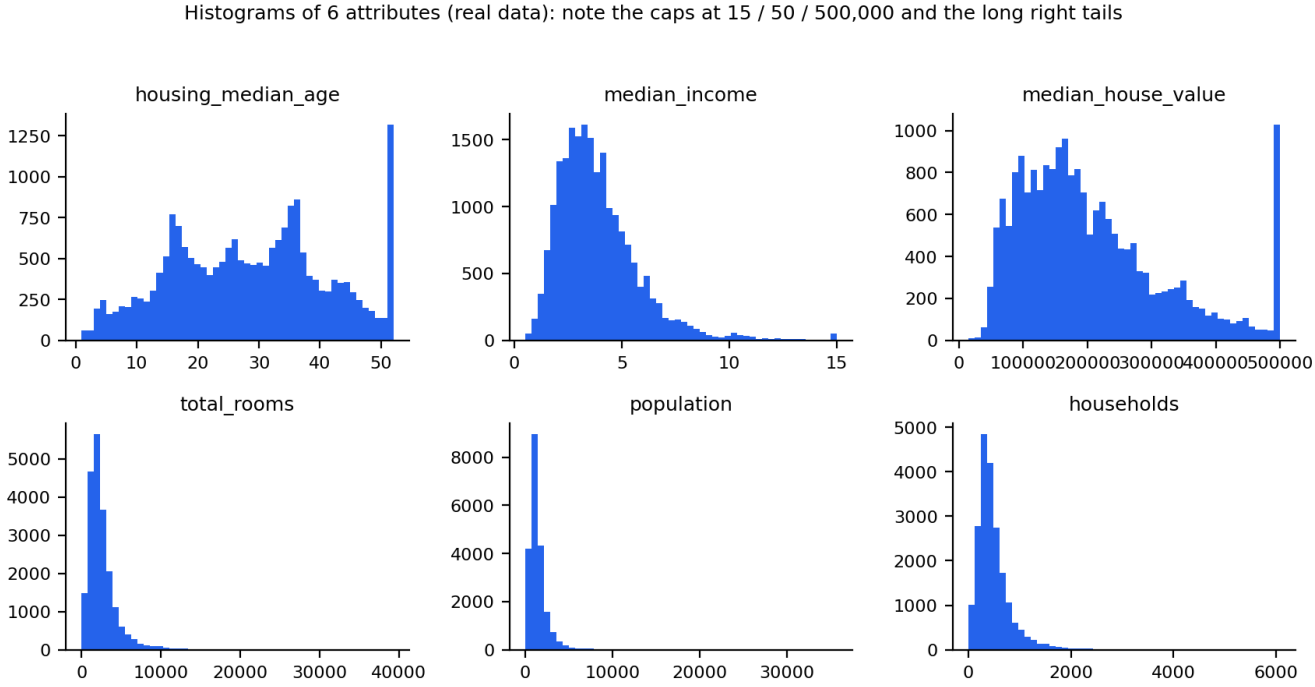
Output (real run)

```
(20640, 10)
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
```

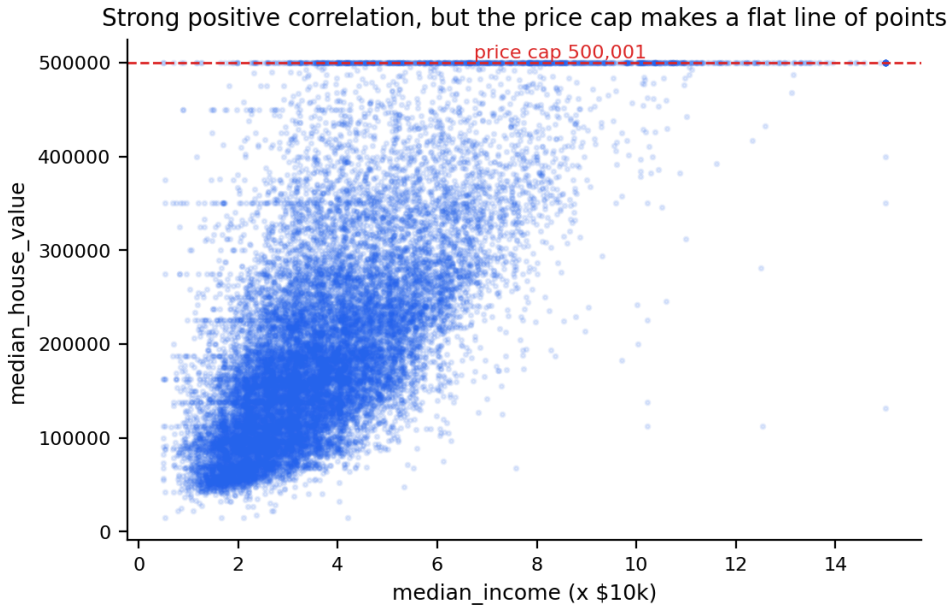
```
ISLAND          5
207 missing values in total_bedrooms
```



Geographic location (real data): each point = a district, its size = the population, and its color = the price. The most expensive (red) is close to the coast and the big cities. This shows that location is a very important feature.



Histograms of 6 columns (real). Note: `housing_median_age` is capped at 52, and `median_house_value` is capped at 500,001, and the remaining features have a "long tail" to the right.



The correlation between income and price is strong, and there is a clear horizontal line at 500,001 (the capped price).

Addition

- **965** districts have a price of exactly 500,001 (the price was capped). This is dangerous: if the model trains on capped prices it will never predict a price above 500 thousand.
- **1273** districts have a house age of 52 (also capped).
- `median_income` is not in dollars: it is a number between roughly 0.5 and 15 (in tens of thousands).
- **A remark from the book:** if the client needs prices above 500 thousand you must either collect real labels for the capped districts, or remove them from the training and test sets.

3 Performance measures (RMSE, MSE and MAE)

From your notebook

First I must choose a **Performance measure**. The usual measure for **Regression** problems is **RMSE** (Root Mean Square Error); it computes the numeric difference between the actual value and the predicted value.

The symbols (from your notebook) and their meaning:

- `m` : the number of instances in the dataset we measure on (number of instances).
- `i` : just an index, not an exponent.
- `h` : the hypothesis, i.e. the model itself (the function that predicts).
- `x(i)` : a vector containing all the feature values of instance number i (excluding the label), like: longitude, latitude, number of rooms... etc.
- `y(i)` : the actual value (Label) for instance i, one number.
- `h(x(i))` : what the model predicts for instance i, and we also write it `ŷ(i)` .

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

From your notebook

RMSE is specific to **Regression** problems only. We do not use it in **Classification** because the result there is a class (a Category), not a number, so we use Accuracy or Precision / Recall / F1-score (especially if there is class imbalance like fraud detection) or a Confusion matrix.

MSE is also specific to Regression and uses the same logic, and most models use it as a loss function. The difference: MSE gives the same error information but squared (its unit is dollars²), so RMSE is more understandable.

$$\text{MSE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}(h(\mathbf{x}^{(i)}) - y^{(i)})^2$$

From your notebook

Although RMSE is the preferred measure in Regression, in some contexts you may prefer another measure. For example if you have many **outliers** you may think of **MAE** (Mean Absolute Error, also called average absolute deviation).

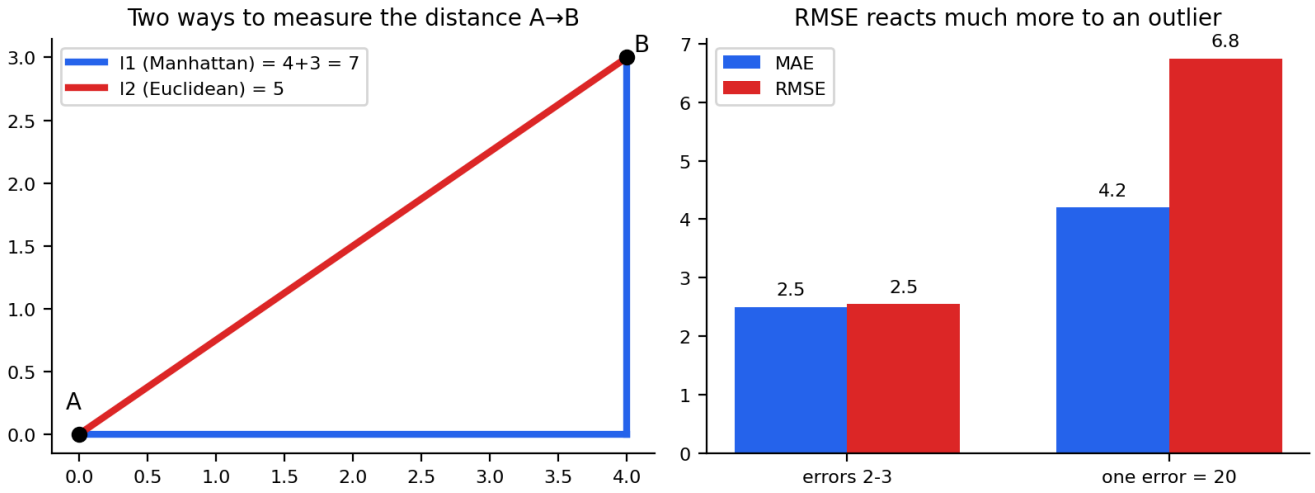
$$\text{MAE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h(\mathbf{x}^{(i)}) - y^{(i)}\right|$$

From your notebook

RMSE and MAE both measure the distance between two vectors: the predictions vector and the true values vector. There is more than one way to define "distance", and these are called **norms**:

- **$\ell 2$ norm (Euclidean norm)** ← RMSE uses it. The same straight-line distance concept we are used to (Pythagoras).

$$\ell_2 : \ \|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$



Left: the same distance with $\ell 2$ and $\ell 1$. Right (computed from errors 2-3, and one error = 20): MAE went 2.5 → 4.2, but RMSE went 2.5 → 6.8.

```
from sklearn.metrics import root_mean_squared_error, mean_absolute_error
rmse = root_mean_squared_error(y_true, y_pred)
mae = mean_absolute_error(y_true, y_pred)
```

From your notebook

- **$\ell 1$ norm (Manhattan norm)** ← MAE uses it. It is named that because it is like moving through the streets of a city of blocks: you cannot go diagonally, you must walk along the length and the width. It sums the absolute values without squaring or roots.

$$\ell_1 : \ \|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n|$$

From your notebook

The most important remark: the higher the norm number, the more the measure focuses on large values and ignores small ones. That is why RMSE is more sensitive to outliers than MAE.

The rule: if the outliers are very rare (like the bell curve) RMSE works very well. If you have many outliers use MAE.

Addition

Generalization: the norm with number `k` is computed like this, and we get $\ell 1$ and $\ell 2$ by changing k:

$$\|\mathbf{v}\|_k = \left(|v_1|^k + |v_2|^k + \cdots + |v_n|^k\right)^{1/k}$$

Addition

Special cases: `ℓ0` counts the number of non-zero elements, and `ℓ∞` gives the largest absolute value in the vector.

Note / Correction

In recent versions of scikit-learn the `squared=False` parameter was removed (it was deprecated in 1.4 and removed in 1.6). The direct replacement is `root_mean_squared_error`. The book's code (its edition) uses `squared=False`, so if you run it you will get an Error; use the replacement. The actual experiment on my version (1.8):

```
mean_squared_error([1, 2], [1, 3], squared=False)
```

Output (real run)

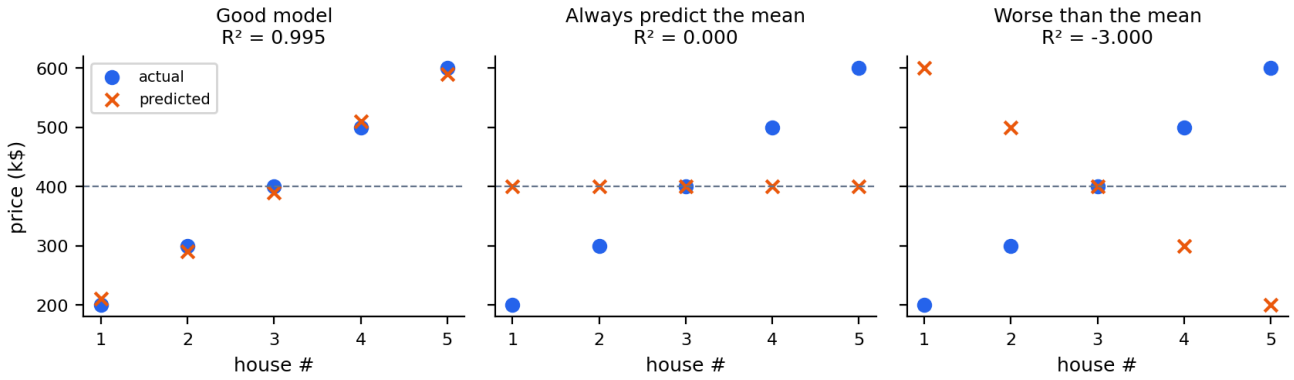
```
TypeError: got an unexpected keyword argument 'squared'
```

4 The R² score metric

```
from sklearn.metrics import r2_score
r2_score([200, 300, 400, 500, 600], [210, 290, 390, 510, 590])
```

Output (real run)

```
0.995
```



The same as your houses: the predictions 210/290/390/510/590 give $R^2 = 0.995$ (actually computed). Predicting with the mean only = 0. A reverse prediction gives a negative value (−3).

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

From your notebook

The **R^2 score** is a metric for evaluating Regression models. Its basic job is to tell you: how much of the variation in the real values of the Target the model was able to explain, compared with predicting the mean only.

Example: we have house prices 200k, 300k, 400k, 500k, 600k. If we had not done ML at all and said "I do not know the price of any house, I will predict the average price" (the mean here = 400k), so every house we say costs 400k. This is considered the **Baseline**. The model uses the features (age, location, bedrooms) to try to know the prices of the different houses. Instead of saying 400k for all, it may say: 210k, 290k, 390k, 510k, 590k. So the question: how much of the variation in house prices did the model manage to explain? This is roughly what R^2 measures.

- **$R^2 = 0$:** the model cannot explain any variance; it is no better than using the mean of the Target.
- **$R^2 = 1$:** the model explains all the variance perfectly; the predictions are identical to the true values (the ideal case).
- **$R^2 = 0.8$:** the model explains about 80% of the variance in the Target compared with the mean baseline.

This does **not** mean the model predicts correctly 80% of the time, nor that the Accuracy = 80%, nor that 20% of the predictions are wrong.

Addition

R^2 can come out **negative** if the model is worse than predicting the mean (like the figure). The fraction in the equation = (sum of squares of the model's errors) ÷ (sum of squares of the errors of the mean baseline), so the smaller the fraction the closer R^2 is to 1.

R^2 is a relative measure with no unit, and RMSE is in the original unit (dollars); they complement each other: R^2 tells you "the model is better than the baseline by how much", and RMSE tells you "the typical error in dollars".

Note / Correction

The name R^2 comes originally from (squared) because in simple linear regression it equals the square of the correlation coefficient r between x and y. The same equation (1 − the ratio of the sums of squares) is its general way of calculation.

5 Creating the Test Set

From your notebook

The idea is simple: take a random part of the data (usually 20%) and put it aside, and we call it the **test set**. This code does it manually:

```
import numpy as np
def split_train_test(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]

from sklearn.model_selection import train_test_split
train_set, test_set = train_test_split(housing_full, test_size=0.2, random_state=42)
print(len(train_set), len(test_set))
```

Output (real run)

```
16512 4128
```

From your notebook

- `test_ratio`: the size of the data that will go to test (for example 0.2).
- `np.random.permutation(len(data))`: returns the numbers of all the rows, but randomly shuffled.
- `test_set_size`: the number of rows of the test set (data × ratio).
- `shuffled_indices[:test_set_size]`: the first part of this size is for test, and the rest `[test_set_size:]` is for train.
- `data.iloc[...]`: returns a new table containing the rows at these positions (shuffled, not in order).

Or I can say (ready for any data): `train_test_split`.

Addition

Why do we isolate the test set early? If you look at it while you choose your model, your brain may catch a pattern in it and bias your choice, and this is called **data snooping bias**, and the result is that your estimate of the real performance becomes over-optimistic.

The problem of updating the data

```
from zlib import crc32
def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]

print([is_id_in_test_set(i, 0.2) for i in range(1, 11)])
```

Output (real run)

```
[False, True, False, False, True, False, False, False, False, False]   # ids 2 and 5 go to the test set - every single run
```

Addition

`random_state=42` fixes the randomness: every time you run the code you will get the same split (the number itself is not important, 42 is just a convention).

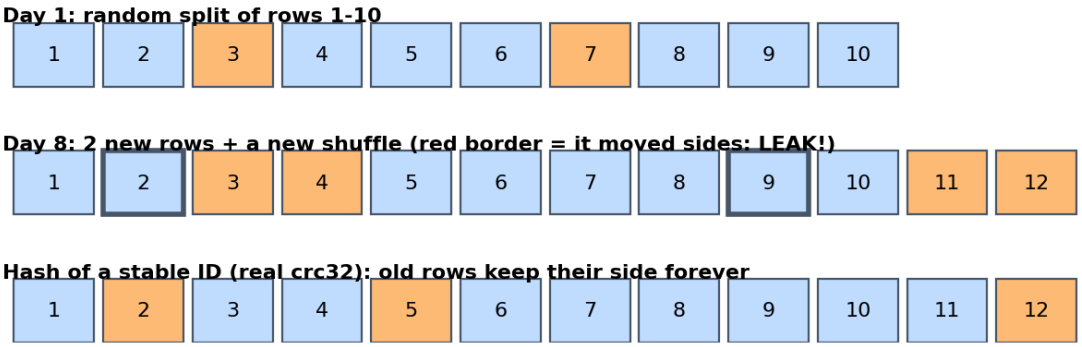
From your notebook

If I split the data randomly, trained and tested, and a week later new data arrived and I added it to the same dataset and re-ran the same split code, a problem happens: even if you fixed `random_state`, `len(data)` itself changed so the order will be completely different. That means some rows that were in train may now go to test (and vice versa). That is a serious problem because the model has seen some of the test data during training (**data leakage**) and this distorts the credibility of the evaluation.

The solution: instead of relying on the "random ordering of the index" I rely on the **identity (ID)** of each row: every row has a fixed ID, and I decide that it goes to test based on the ID itself, not on its order in the file. So even if the data grows or its order changes, the row goes to the same set.

How do I decide? With a **hash function** like `crc32`: it takes any input (number/text) and returns a fixed number. The same input always gives the same output (the opposite of randomness), and the output number is distributed in a nearly random way over a wide range of numbers. That means `crc32(5)` will always return the same number no matter how many times you run the code, and this is the essential difference from `np.random` which changes every time.

$$\text{crc32(id)} < \text{test_ratio} \times 2^{32} \Rightarrow \text{test set}$$



Repeated random splitting makes rows move between train and test (red border = leak), but the hash of a fixed ID keeps the row on its side. The last line is from the real crc32: ids 2, 5 and 12 are in the test set.

From your notebook

The first function takes the row's identity and computes its hash (a number between 0 and 2^{32}), and if it is less than `test_ratio * 2**32` (i.e. the first 20% of the range) it returns `True` (the row goes to test) and otherwise `False`. The second function applies this to every row: those with `True` go to test, and those with `False` (`~in_test_set`) go to train.

Addition

The ratio will not come out exactly 20% in a small sample (here 2 out of 10 happened to be 20%), but with large data it gets very close to it.

From your notebook

Now `split_data_with_id_hash` needs an ID column, and `housing` does not have one.

The first solution: use the row number itself as the ID:

```
housing_with_id = housing_full.reset_index()      # adds an "index" column
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

From your notebook

`reset_index()` takes the DataFrame's internal index and turns it into a normal column named `index`.

The problem with this solution: the row number is not a "real ID" of the house, it is just its order in the table. If you delete a row from the middle (not from the end) all the rows after it change their number (move up one), so the ID of the same thing changes although it did not change. That ruins the whole idea. (New data must be added at the end and no row deleted.)

The second solution: use columns from the data itself to build an ID (for example the geographic location):

```
housing_with_id["id"] = housing_full["longitude"] * 1000 + housing_full["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

From your notebook

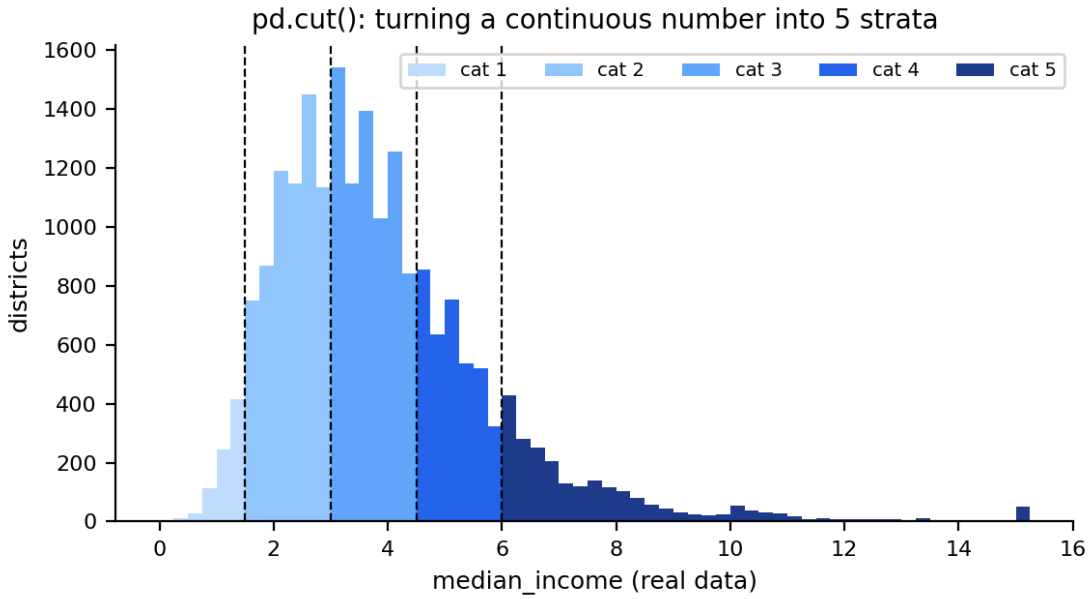
The important thing is the idea: a new column named `id` formed from fixed features unique to each row.

Note / Correction

The second solution has a real trap I tried: geographic location is **not unique**! On this data I found **8,050** rows (39% of the data) share the same `id` with another row (many districts with exactly the same coordinates). This means these groups will all go together to train or test, so the split is "by groups" and not by rows, and this reduces the randomness of the sample a bit. You must know this: the ideal ID is **unique and stable**.

6 Stratified Sampling

```
housing_full["income_cat"] = pd.cut(housing_full["median_income"],
                                    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                    labels=[1, 2, 3, 4, 5])
```



The dashed lines = the bin boundaries, and each color = a category (from the real data).

From your notebook

Imagine a box with 10 balls: 6 red and 4 blue (i.e. the ratio 60% / 40%). We want to draw 4 balls only from the box (a small sample) and we want this sample to represent the whole box.

The first method: simple random draw. The 4 balls we draw might not come out in a ratio close to 60/40, and they might all come out red by chance (i.e. worry about luck).

The second method: Stratified Sampling: instead of drawing randomly from the whole box, we do:

1. Separate the balls into two groups: a red pile and a blue pile.
2. I want to draw 4 balls out of 10, i.e. 40% of the box.
3. Apply the same 40% to each pile separately.

The goal: when I have an important category in my data, Stratified guarantees that the train/test split takes each category with the same proportion as the real one, instead of a random draw leaving it to chance, which may produce a non-representative sample.

From your notebook

But if the feature is a continuous number (like `median_income`) there are no clear "groups" to split on. **The solution:** I make groups (**strata**) myself with `pd.cut()`:

`pd.cut()` takes a continuous numeric column and divides it into categories according to the boundaries (`bins`): from 0 to 1.5 → category 1, from 1.5 to 3 → category 2, and so on. And the `labels` are the names of the categories. The result: a new column `income_cat` and each house now has a "category" (from 1 to 5) instead of the decimal number. These are the strata.

```
strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2, stratify=housing_full["income_cat"], random_state=42)
```

```
for set_ in (strat_train_set, strat_test_set):     # remove income_cat so it is never used as a feature
    set_.drop("income_cat", axis=1, inplace=True)
```



The green columns (stratified) are stuck to the truth, and the orange (random) drifts a bit (from the real run).

From your notebook

Now I split the data while keeping the proportion of each category. There are two ways, one long and one easy, which is: it is exactly `train_test_split` that we talked about before, but adding `stratify`. After we used `income_cat` we must delete it from train and test so that it does not wrongly enter as a feature in the model training.

Addition

A real result: the proportion of each `income_cat` in all the data versus the test set (stratified vs random):

income_cat	% Overall	% Stratified	% Random
1	3.98	4.00	4.02
2	31.88	31.88	32.44
3	35.06	35.05	35.85
4	17.63	17.64	16.74
5	11.44	11.43	10.95

Addition

The largest relative deviation from the truth: random **5.1%** versus stratified **0.36%**. Why income in particular? The book says median income is the most important factor for predicting the house price, so it is important that the test set represents all the income categories in their real proportions.

7 Feature engineering and correlation analysis

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]

corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

Output (real run)

```
median_house_value    1.0000
median_income         0.6872
rooms_per_house       0.1463
total_rooms           0.1351
housing_median_age    0.1141
households            0.0646
total_bedrooms        0.0478
people_per_house     -0.0220
population           -0.0269
longitude            -0.0475
latitude             -0.1427
bedrooms_ratio       -0.2600
```

From your notebook

Feature engineering: extract useful information by merging or transforming existing columns (like the division between them) so that I reach a new column that is more correlated and indicative of what I am trying to predict, instead of relying on the raw columns alone. And make sure it is useful by comparing `corr` before and after.

Addition

From the book: `median_income` ≈ 0.688, `rooms_per_house` ≈ 0.144, and `bedrooms_ratio` ≈ −0.256. In the numbers from my run above: `median_income` = 0.687, `rooms_per_house` = 0.146, `bedrooms_ratio` = −0.260.

`bedrooms_ratio` (negative: the lower the ratio of bedrooms to total rooms, the more the price tends to rise) is more informative than the raw `total_rooms` and `total_bedrooms`.

Reminder: `corr()` measures only the **linear** relationship; a strong non-linear relationship may come out with a small value.

8 Missing values (SimpleImputer) and Outliers

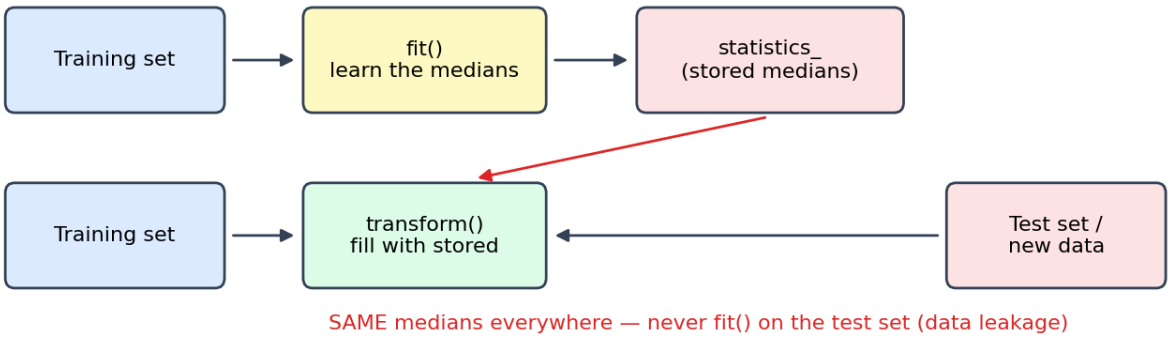
From your notebook

To deal with the `missing values` in the data after the drop, I fill the places of these missing values with a suitable value. **SimpleImputer** = "the tool that fills missing values". It is a ready class from sklearn doing the same idea as Pandas' `fillna()`, but in a standard way and designed to integrate with the rest of the sklearn tools.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
housing_num = housing.select_dtypes(include=[np.number])    # the median only works on numbers
imputer.fit(housing_num)
print(imputer.statistics_)          # the median learned for each column
X = imputer.transform(housing_num)  # really fills the missing values
```

Output (real run)

```
[-118.51   34.26   29.00 2119.00  433.00 1164.00  408.00    3.54]
```



`fit` on the training set only, then `transform` on anything else with the saved values.

From your notebook

`strategy="median"` : tells the tool when it finds a missing value, replace it with the **median** of the column; exactly like `housing["total_bedrooms"].fillna(median)` , but here it computes it for all the columns.

I used `SimpleImputer` instead of `fillna()` because it is designed to work inside a **pipeline** and to save the value it learned in `fit()` , so we can apply it automatically on new data without recomputing the median each time.

- `fit()` : means "learn from this data": computes the median of each column and saves it without changing the data itself.
- `transform()` : now apply what it learned on the data (fill all the values with the saved median).

Why separate `fit` and `transform` ? Because this is the foundation of sklearn's philosophy: `fit` on the **training data only** (learn the median), and after that `transform` on any other data (like the test set) with the median learned from the training, not computing a new median from the test (this is called data leakage between train and test).

Remark: `transform()` returns a NumPy array, and if you want a DataFrame:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns, index=housing_num.index)
```

Addition

In the training set we have **158** missing values in `total_bedrooms` (the other columns have none), and `SimpleImputer` fills them with the median (433).

The strategies available in `SimpleImputer` : `"mean"` (the default), `"median"` , `"most_frequent"` (it works with text as well), `"constant"` (a value you specify yourself with `fill_value`). We will use `most_frequent` with text columns later in section 14.

Detecting outliers with Isolation Forest

```
from sklearn.ensemble import IsolationForest
isolation_forest = IsolationForest(random_state=42)
outlier_pred = isolation_forest.fit_predict(X)
print((outlier_pred == -1).sum(), "outliers out of", len(X))

# if you want to drop them after identifying them:
housing = housing.iloc[outlier_pred == 1]
housing_labels = housing_labels.iloc[outlier_pred == 1]
```

Output (real run)

```
2116 outliers out of 16512
```

From your notebook

An algorithm that discovers the "strange" rows automatically and returns `-1` for the outlier row and `1` for the normal row.

9 Encoding text columns

```
from sklearn.preprocessing import OrdinalEncoder
ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)     # housing_cat = the text column
print(housing_cat_encoded[:5].ravel(), housing_cat.values[:5].ravel())
print(ordinal_encoder.categories_)
```

From your notebook

ML models are mathematical algorithms that only understand numbers, not text (i.e. if you have a column like `INLAND` I must convert it to numbers in some way).

The first solution: `OrdinalEncoder` — The idea: each text category takes and gets a number (0, 1, 2, ...). `fit_transform()` : a shortcut for `fit` and `transform` together in one line.

Output (real run)

```
[1.0 4.0 1.0 4.0 0.0] ['INLAND' 'NEAR OCEAN' 'INLAND' 'NEAR OCEAN' '<1H OCEAN']
[['<1H OCEAN' 'INLAND' 'ISLAND' 'NEAR BAY' 'NEAR OCEAN']]
```

From your notebook

The problem: when it gives `INLAND = 1` and `ISLAND = 2` for example, the model will understand that 1 and 2 are close, and 1 and 4 are far, as if there were a real logical ordering between the categories, and this assumes a wrong relationship between the categories.

From your notebook

The more correct solution: `OneHotEncoder` — The idea: instead of giving each category a number, we create a separate column for each category and put `1` in the column representing the right category and `0` in all the rest. This way the model does not assume any ordering or closeness between the categories; all of them are equal and completely separate.

```
from sklearn.preprocessing import OneHotEncoder
cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
print(housing_cat_1hot.shape, type(housing_cat_1hot).__name__)
print(housing_cat_1hot.toarray()[:3])
```

Output (real run)

```
(16512, 5) csr_matrix
[[0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 0.]]
```

Category	OrdinalEncoder	OneHotEncoder
<1H OCEAN	0	[1, 0, 0, 0, 0]
INLAND	1	[0, 1, 0, 0, 0]
ISLAND	2	[0, 0, 1, 0, 0]
NEAR BAY	3	[0, 0, 0, 1, 0]
NEAR OCEAN	4	[0, 0, 0, 0, 1]

Addition

The Ordinal problem: 0 and 1 "close", 0 and 4 "far" without a logical reason. In our case `ocean_proximity` does not really have a true ordering at all (except ISLAND which is its own thing).

`pd.get_dummies()` is easy but it "does not remember" the categories it saw, so if a new category appears in the later data it will make different columns. As for `OneHotEncoder`, it saves the categories in `fit()` and you can control the new category with `handle_unknown="ignore"` (we use it later in the pipeline).

If the number of categories is very large, one-hot will make a huge number of columns; then you think of alternatives like **embeddings** (explained in Deep Learning).

From your notebook

A strange thing: when I come to display `housing_cat_1hot` I find that most of the values in the table are zeros, because each row has only one number `1` out of 5 columns. sklearn returns the output as a **SciPy sparse matrix** instead of a normal NumPy array, meaning it stores only the positions of the non-zero numbers (the 1s), not all the zeros. This saves a lot of memory. (Imagine you have 1000 categories: each row would store 999 zeros for nothing.)

```
housing_cat_1hot.toarray()                      # to see it as a normal array (with all the zeros)
cat_encoder = OneHotEncoder(sparse_output=False) # or ask for a dense array from the start
cat_encoder.feature_names_in_                   # the names of the input columns
cat_encoder.get_feature_names_out()             # the names of the new columns after encoding
```

Output (real run)

```
['ocean_proximity_<1H OCEAN' 'ocean_proximity_INLAND'
 'ocean_proximity_ISLAND' 'ocean_proximity_NEAR BAY'
 'ocean_proximity_NEAR OCEAN']
```

Useful for building a clear DataFrame with understandable column names instead of abstract numbers.

10 Feature Scaling

From your notebook

The process that takes all your columns (Features) and puts them all in a close or unified range, instead of each column having a completely different scale from the other.

The problem: if the first column ranges from 0 to 40 thousand (a very wide range) and the second from 0 to 20 (a very narrow range). Most of the model algorithms compute mathematical calculations that depend on the differences between numbers. If I fed the model these numbers as they are, the first column will dominate because its numbers are larger, not because it is more important. So we put all the columns in the same "numeric playground", and the model compares between the columns fairly.

The first solution: Min-Max Scaling

```
from sklearn.preprocessing import MinMaxScaler
min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
print(housing_num_min_max_scaled.min(axis=0).round(1), housing_num_min_max_scaled.max(axis=0).round(1))
```

$$x_{\text{scaled}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

Output (real run)

```
[-1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0] [1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0]
```

From your notebook

The result: the smallest value in the column is always 0 (because `x - x_min = 0`), and the largest value is 1 (because the numerator = the denominator), and all the other values fall between them.

Sometimes we change the default range (0, 1) (we can specify a different range like (−1, 1)) if our model prefers values around zero (like neural networks in Deep Learning). The equation is modified a bit to reach the new range, but the basic idea is the same.

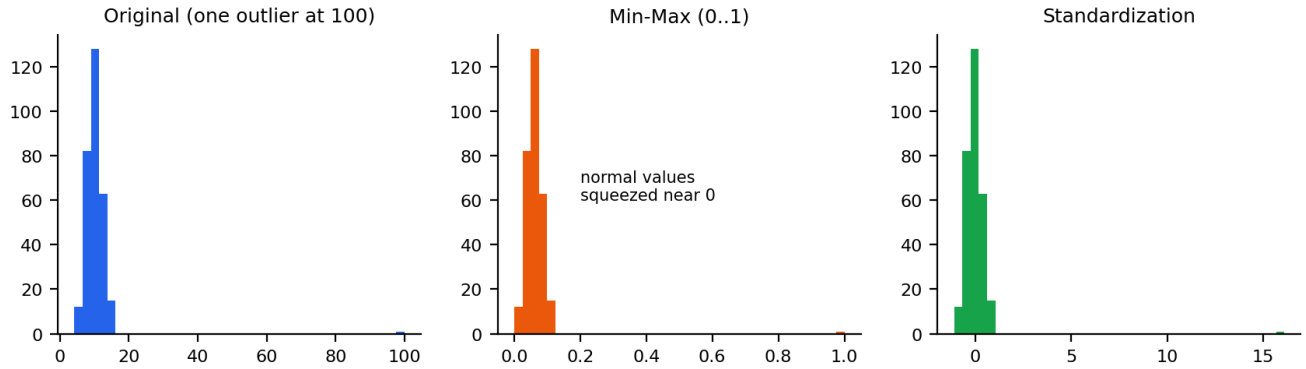
The second solution: Standardization (stronger with outliers)

```
from sklearn.preprocessing import StandardScaler
std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
print(housing_num_std_scaled.mean(axis=0).round(2), housing_num_std_scaled.std(axis=0).round(2))
```

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Output (real run)

```
[ 0.00  0.00  0.00 -0.00 -0.00 -0.00 -0.00 -0.00] [1.00 1.00 1.00 1.00 1.00 1.00 1.00 1.00]
```



Data with one outlier (100) (synthetic, real run): Min-Max squeezed the normal values between 0.00 and 0.12 only, while Standardization kept a better spread (−1.1 to 0.9).

From your notebook

This means: (the value − the column mean μ) ÷ (the standard deviation σ).

The essential difference from Min-Max: here there is no fixed range (0 to 1), but the column mean becomes 0 and the standard deviation 1. And the values can come out as any number (positive or negative), not bounded by a certain limit.

Why is it stronger with outliers? If all the data is from 0 to 20 and an entry error happened and 100 (or 200) was written: with Min-Max this large value will be the `max`, so all the rest of the normal values (0 to 20) will turn into very small values close to zero (because `x_max - x_min` became huge). But in Standardization the mean and the standard deviation are only affected a little, so the result is better.

The same rule as SimpleImputer: any Scaler must `fit()` on the training set only, and then `transform()` on the test set and any new data with the same scaler. We do not do `fit()` on the test set.

Addition

When do I use which?

- Min-Max: when I need a specific range (like 0..1 or −1..1, especially for neural networks).
- Standardization: when the data has outliers or I do not want a fixed range.

Models that depend on trees (Decision Tree / Random Forest) are generally **not sensitive** to the scale of the features, but linear models, KNN and neural networks benefit a lot from scaling.

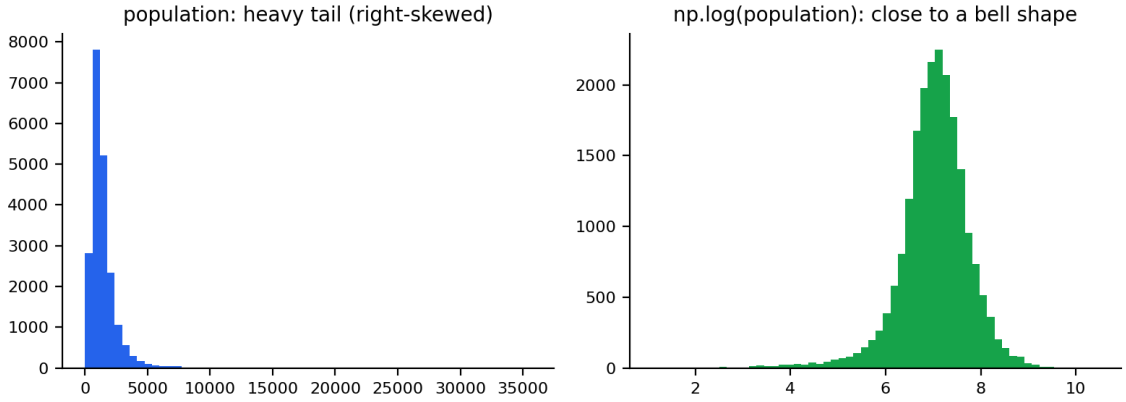
11 Feature transformations (Log, Bucketizing and RBF)

Log Transform

From your notebook

The problem: some features have a "heavy tail" distribution; i.e. most of the values are gathered in a small range (like most neighborhoods have 1,000 to 3,000 people), but in a very small number of extreme cases with huge numbers (10,000 or 20,000 people). So the shape of the distribution appears as a long tail extending to the right (right-skewed). Models work better when the shape of the data distribution is close to the normal distribution (Gaussian / bell-shaped). If the column is "heavy tail" models struggle to learn from it properly.

```
import numpy as np
housing["population"] = np.log(housing["population"])    # the name of the column you want to take the
log of
print(housing_full["population"].skew(), np.log(housing_full["population"]).skew())
```



The distribution of the real `population` before and after the logarithm.

From your notebook

The solution: take the logarithm of the column. The logarithm compresses the big differences (between 10000 and 20000 for example) and expands the small differences, so the final result gets very close to the ideal bell shape.

```
Output (real run)
4.936 -1.071      # skewness: +4.9 (long tail) -> -1.1 (almost symmetric)
```

Addition

The logarithm works on **positive values only** (`log(0)` or of a negative number is undefined). If there are zeros use `np.log1p`.

From the book: if the heavy tail is on the right, the first thing you can try is the square root (or raising to a power between 0 and 1). And the logarithm is used when the tail is very long (power law) like `population`. In the project the log was applied to the columns: `total_bedrooms, total_rooms, population, households, median_income`.

Bucketizing

From your notebook

The basic idea: instead of keeping the original value (a continuous number), we divide the range of values into "buckets / bins" of roughly equal size, and replace each value with the number of the bucket it fell in (not the value itself). It is similar to the idea I used in Stratified sampling with `pd.cut`, but the goal is different: here we use it on a `feature` to give it to the model (the stratified division was for the split).

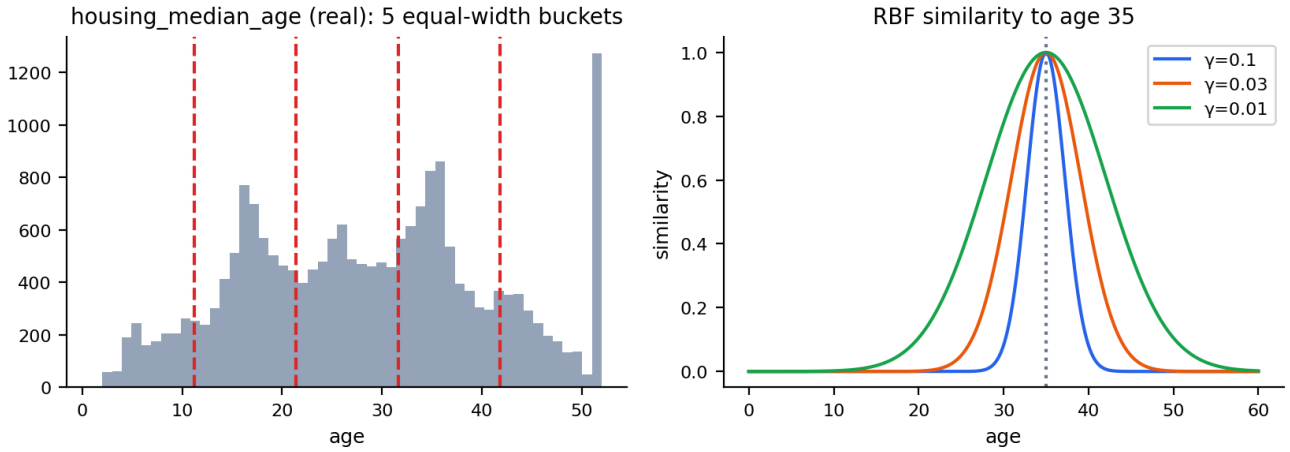
Example: if I have a column ranging from 0 to 15 and I divided it into 5 equal buckets (equal-sized buckets):

The first case: when each bucket has roughly the same number of rows (like the first 20% of the values go to bucket 1, the second 20% bucket 2...) the result is that the column becomes almost uniformly distributed (uniform), and there is no such thing as scaling for a column like this at all.

The second case: when the column is **Multimodal** (i.e. its distribution has more than one peak (modes), not a single bell shape). Example: house ages cluster around 20 years and another cluster around 35 years. Bucketizing in this case makes the bucket number treated as a category (not a normal number) and we use `OneHotEncoder` on it. So the model treats each bucket as a completely separate category, and learns a different behavior for each category. This is the best in the multimodal distribution case because numeric closeness is not always useful (bucket 3 is "close" to bucket 4 numerically, but it may resemble bucket 1 more in behavior).

Addition

The book's multimodal example is `housing_median_age`: in the real data we have a strange peak at 52 (1273 districts, that is the cap), and other peaks around 35 and 36 (824 and 862 districts). Houses built around 35 years ago may be an old style that went out of fashion, so their prices are cheaper, which suggests their age on its own. OneHot on the buckets allows the model to learn a different rule for each age range. And RBF gives you another way: a feature for each main peak (mode).



Left: 5 equal-width buckets on the real `housing_median_age`. Right: the RBF curve of similarity to 35 with different γ values (larger γ = narrower and faster drop).

RBF-Similarity Features

From your notebook

The idea: imagine you want to know "how close a certain number is to another fixed number" (not "is it bigger or smaller"). For example: you want to know how close the house age is to 35 years. If the age is 35, the closeness = 1 (maximum degree), if the age is 36 or 34 the closeness = 0.9, if the age is 10 or 60 the closeness = 0.1 or less (very far). This is exactly what the **Radial Basis Function (RBF)** computes: a measure of "closeness/similarity" between a certain value and an important fixed point.

Why do we need this? As we said, if we found that in a large cluster of houses their age is around 35 and their price is cheap (for example), instead of doing bucketize (separate categories) we can do something more precise: create a new feature that says "how close the house is to the age of 35" more precisely. So instead of the ordinary age column we get a new column named "similarity to age 35" ranging between 0 (very far) and 1 (exactly 35).

```
from sklearn.metrics.pairwise import rbf_kernel
age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
# on 5 sample ages (35, 34, 36, 10, 60):
print(rbf_kernel(pd.DataFrame({"housing_median_age": [35, 34, 36, 10, 60]}), [[35]],
gamma=0.1).ravel().round(3))
```

Output (real run)

```
[1.000 0.905 0.905 0.000 0.000]
```

From your notebook

`gamma` : a coefficient that controls the "speed" of the decrease in similarity the further we get from 35.

The mathematical equation:

$$\text{similarity} = \exp\left(-\gamma(x-35)^2\right)$$

From your notebook

`exp` is the exponential function: it takes a large negative number and turns it into a number close to zero, and the closer the number inside `exp` is to zero (i.e. `x` close to 35) the output is close to 1.

Addition

From the book: the RBF we measure similarity to 35 with reaches its maximum value (1) at 35, and that is exactly next to the spike in the distribution of `housing_median_age` . And the most used RBF is the Gaussian RBF. And notice that the result above matches the equation: the ages 34 and 36 = 0.905 (not exactly 0.9: that is approximate).

12 Transforming the Target and Custom Transformers

```
from sklearn.compose import TransformedTargetRegressor
from sklearn.linear_model import LinearRegression
model = TransformedTargetRegressor(LinearRegression(), transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

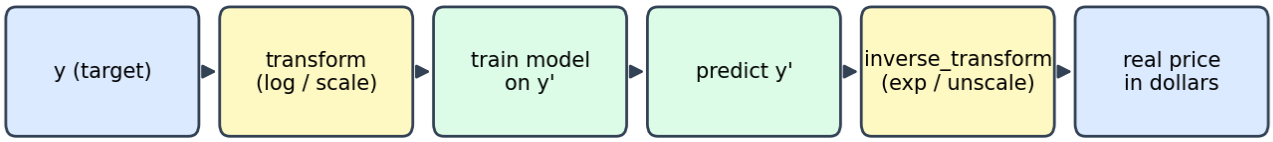
`TransformedTargetRegressor` transforms the target for training and automatically returns it to its real value.

From your notebook

Bucketizing, RBF, Log transform and scaling were all applied to the `features`. Now the topic is different: we are talking about transforming the **target** itself (the `label`; the value the model is trying to predict).

Why do I transform the target? For the same reason as the features: if the distribution of the `target` itself is "heavy tail" we may transform it (like the logarithm) so that training is easier and more accurate for the model. But a new problem appears: if you took the logarithm of the target (for example the logarithm of the median price), the model will predict the logarithm of the price, not the price itself. So now I need to do an **inverse transform** (the inverse function, i.e. `exp`) to bring back the real price in dollars.

The solution: there are two ways: the first is manual and the second is better and more accurate:



Custom Transformers

```
from sklearn.preprocessing import FunctionTransformer
# Ex 1
log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
# Ex 2
rbf_transformer = FunctionTransformer(rbf_kernel, kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

From your notebook

`FunctionTransformer`: its idea: if you want to do a simple transformation (like log and/or RBF).

The real problem: all the tools we used before (`StandardScaler`, `SimpleImputer`, `OneHotEncoder`) all have the same shape: `fit()` and `transform()` and `fit_transform()`. And this is not a coincidence; it is so that we can chain them together in a `Pipeline`. If you used `np.log()` directly it just returns an array and has no fit/transform, so it will not be put in a Pipeline and will not combine with the other tools.

`FunctionTransformer` wraps any function in a transformer with the same shape as the sklearn tools, so you can put it in a pipeline and combine it with others, and the pipeline runs everything automatically in order.

Addition

If the transformation needs to really learn something from the data (a real `fit()`), like the mean in `StandardScaler` or the cluster centers in `KMeans`, you write a class that inherits from `BaseEstimator` and `TransformerMixin` and define `fit()` and `transform()`. The book does this in `ClusterSimilarity` (RBF + KMeans), and here is the full code:

```
class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters; self.gamma = gamma; self.random_state = random_state
    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, n_init=10, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self        # fit() must return self
    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)
    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in range(self.n_clusters)]

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]])
print(similarities.shape); print(similarities[0].round(2))
```

Output (real run)

```
(16512, 10)
[0.00 0.16 0.82 0.00 0.00 0.00 0.00 0.00 0.47 0.34]
```

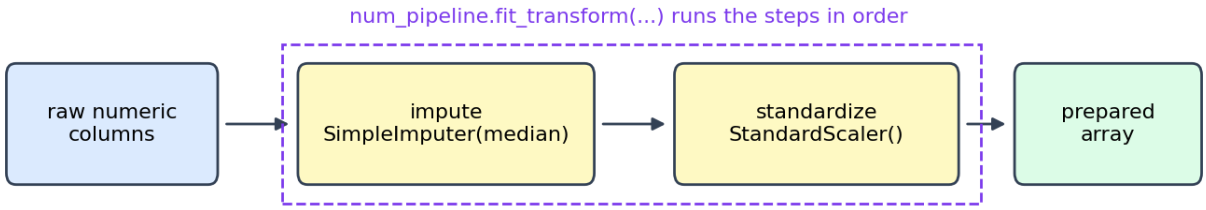
The idea: KMeans finds 10 geographic centers (population clusters), then we measure the similarity of each district with each center with RBF. The result is 10 new `features` telling the model "this district is this close to such-and-such center".

13 Pipelines

```
from sklearn.pipeline import Pipeline
num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])

from sklearn.pipeline import make_pipeline
num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())

housing_num_prepared = num_pipeline.fit_transform(housing_num)    # one line runs all the steps
print(housing_num_prepared.shape, [name for name, _ in num_pipeline.steps])
```



The Pipeline runs the steps in order and passes the output of each step to the one after it.

From your notebook

The problem: I now have multiple steps (`SimpleImputer` , filling the missing values; `StandardScaler` , unifying the scale), each separate, and I must run them one by one and pass the output of one to the other. If you have a long chain of steps (like 5-6 steps) you will have to write them one by one and follow their sequence by hand, which is error-prone and not organized.

The solution: Pipeline — it takes a list of steps in order, each step is a (name, tool). When you do `fit_transform()` on the whole Pipeline it runs the steps automatically in order: first `SimpleImputer` (fills the missing values) and its output is automatically passed to `StandardScaler` (unifies the scale). The names (`"impute"` and `"standardize"`) are optional, they are labels to clarify what each step does.

An easier version: `make_pipeline` — exactly the same idea but it gives automatic names instead of you writing them:

Output (real run)

```
(16512, 8) ['simpleimputer', 'standardscaler']
```

ColumnTransformer

```
from sklearn.compose import ColumnTransformer
num_attribs = ["longitude", ..., "median_income"]     # numeric attributes
cat_attribs = ["ocean_proximity"]                     # categorical attributes

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])

from sklearn.compose import make_column_selector, make_column_transformer
preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

From your notebook

The problem: the Pipelines we saw before are applied to all the columns in the same way. But the data has different columns: numeric columns (`median_income` and others) and text columns (`ocean_proximity`). The numeric ones need `SimpleImputer` + `StandardScaler` , but the text ones need a different `SimpleImputer` strategy (the most frequent) + `OneHotEncoder` . I must process them separately.

From your notebook

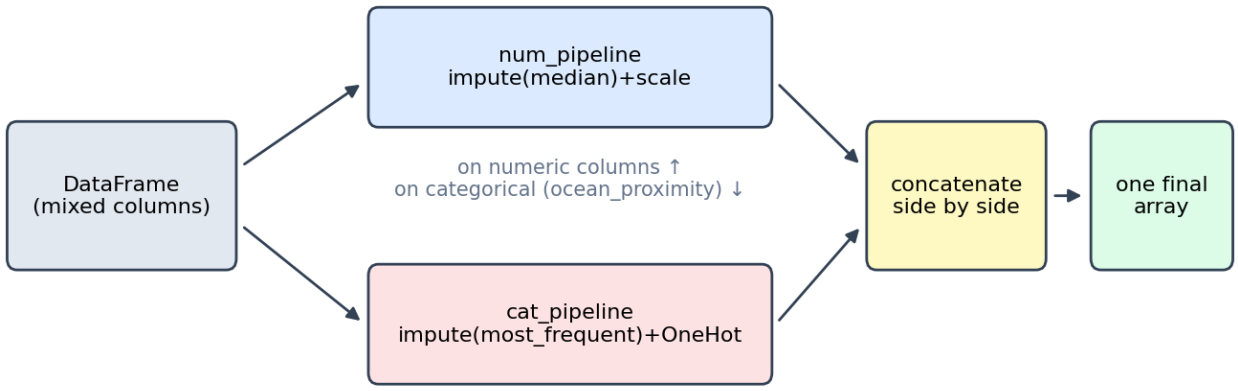
The idea: you specify for each group of columns (a list of names) which pipeline is applied to it. The `ColumnTransformer` runs every pipeline on its assigned columns, and at the end it pastes all the results side by side in one final table.

Here we cannot use `median` because text has no mathematical median; so we used the most frequent category.

`make_column_selector` : choosing the columns automatically by type:

From your notebook

Instead of writing the names of the numeric and text columns manually (and the column names or the data may change), it automatically selects any numeric or text column (more flexible).



ColumnTransformer: each type of column has its own pipeline, and the result gathers in one array.

The full Preprocessing

From your notebook

The following code gathers everything that was done in the project in one place:

```
def column_ratio(X): return X[:, [0]] / X[:, [1]]
def ratio_name(function_transformer, feature_names_in): return ["ratio"]
```

```
def ratio_pipeline():
    return make_pipeline(SimpleImputer(strategy="median"),
                         FunctionTransformer(column_ratio, feature_names_out=ratio_name),
                         StandardScaler())

log_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                             FunctionTransformer(np.log, feature_names_out="one-to-one"),
                             StandardScaler())
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())

preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population", "households",
"median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
], remainder=default_num_pipeline)    # any column not mentioned (housing_median_age) gets impute + scale

housing_prepared = preprocessing.fit_transform(housing)
print(housing_prepared.shape)
print(preprocessing.get_feature_names_out()[:5], "...", preprocessing.get_feature_names_out()[-3:])
```

Output (real run)

```
(16512, 24)
['bedrooms__ratio' 'rooms_per_house__ratio' 'people_per_house__ratio'
 'log__total_bedrooms' 'log__total_rooms'] ... ['cat__ocean_proximity_NEAR BAY'
'cat__ocean_proximity_NEAR OCEAN'
 'remainder__housing_median_age']
```

From your notebook

- `ratio_pipeline`: `SimpleImputer` + `FunctionTransformer` (the division) + `StandardScaler`.
- `cluster_simil`: computes the similarity of each row with the cluster centers (with RBF + KMeans clustering) on `latitude` / `longitude`.
- `remainder=default_num_pipeline`: any column nobody mentioned in any transformer gets the default (impute + scale).
- **The result:** one line `fit_transform` applies all the preprocessing steps we talked about before on all the data.

Addition

The final output after `preprocessing` is `(16512, 24)`: i.e. 16,512 rows (80% of the 20,640) and 24 features: 3 ratios + 5 log + 10 clusters + 5 one-hot + 1 `housing_median_age` = 24.

14 Training real models

From your notebook

The first actual moment in which we train real models after all the preparations.

① Linear Regression

```
from sklearn.linear_model import LinearRegression
lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)              # housing_labels = the real values the model must
predict
housing_predictions = lin_reg.predict(housing)

from sklearn.metrics import root_mean_squared_error
lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
print(housing_predictions[:5].round(-2)); print(housing_labels.iloc[:5].values); print(lin_rmse)
```

Output (real run)

```
[ 70700. 288200. 186000. 189700. 270700.]
[ 72100. 279600.  82700. 112500. 238300.]
69240.0
```

From your notebook

You see part of `housing_predictions` and part of `housing_labels`: some predictions are wrong by a large amount but the rest of the predictions are closer. That is, the model works but is not very accurate. The function `root_mean_squared_error` returns RMSE directly (i.e. it takes the square root of the MSE) instead of the MSE.

The RMSE result is relatively large, so it is **Underfitting**: the model is very simple compared with the complexity of the problem, not able to capture the real pattern of the data.

Addition

From the book: RMSE ≈ 68,688 dollars. In my real run on this data: **69,240** (close; the version of the data we loaded is not 100% identical to the book's version), while most `median_house_value` values are between 120,000 and 265,000, so an error like this is big.

The book gives 3 solutions for underfitting: (1) a stronger model, (2) better features, (3) reduce the constraints (regularization). This model is not regularized, so solution 3 is excluded, so we try a more complex model.

From your notebook

② Decision Tree

The same idea but with a stronger and more complex model (explained in Ch6).

```
from sklearn.tree import DecisionTreeRegressor
tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
tree_rmse = root_mean_squared_error(housing_labels, tree_reg.predict(housing))
print(tree_rmse)
```

Output (real run)

```
0.0
```

From your notebook

When I compute RMSE with the same `lin_rmse` code I find `0`, of course. **The cause:** we are testing the model on the same data it trained on, so of course it memorized it and gives a perfect fake result (error zero!). This is similar to overfitting. We must test the model on data it has not seen before.

The problem: we cannot use the test set now, because the golden rule says do not touch it until you are ready to launch the final model. So we need to evaluate the model honestly without bringing the test set. **The solution:** Cross-Validation.

Cross-Validation (k-fold)

```
from sklearn.model_selection import cross_val_score
tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
print(tree_rmses.mean(), tree_rmses.std())
```

Output (real run)

```
67431.0  3437.1
```

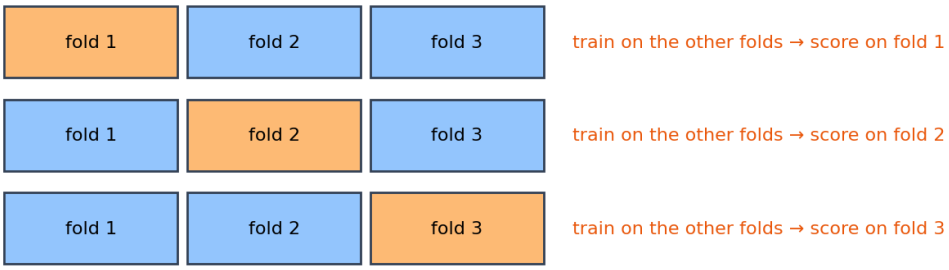
From your notebook

The idea: instead of splitting the data one time only (Train/Validation), Cross-Validation splits the training set into 10 equal parts (**folds**) and does 10 times:

1. We take one part as a "temporary test" (Validation).
2. Train the model on the other 9 parts.
3. Measure the performance on the excluded part.

And the process repeats 10 times, each time with a different part as the "test", and at the end it returns 10 different RMSE results.

Why is this better? Because it is more comprehensive and more truthful than relying on one split, which might come out lucky or unlucky by chance. And you can also compute the standard deviation of the 10 results, and this gives you an idea of how stable the model's performance is, not only its mean.



The idea of k-fold (the figure uses 3 folds for simplicity): each round excludes a different fold for evaluation.

Addition

From the book: the mean RMSE of the Decision Tree in cross-validation ≈ 66,868. Mine: **67,431** (± 3,437). That means it is not better than Linear Regression approximately, and this confirms that the 0 on the train was overfitting.

From your notebook

A remark (the missing point): `cross_val_score` expects a "utility function" (the more it increases the better), the opposite of RMSE which is a cost function (the more it decreases the better). That is why sklearn returns the value negative (`neg_root_mean_squared_error`), and we put a negative sign in front of it at the start to flip it.

The result is like the book's example: a very big drop (really overfitting), and this is exactly the reason for the importance of Cross-Validation: it exposed the truth which the naive evaluation was hiding.

From your notebook

③ Random Forest

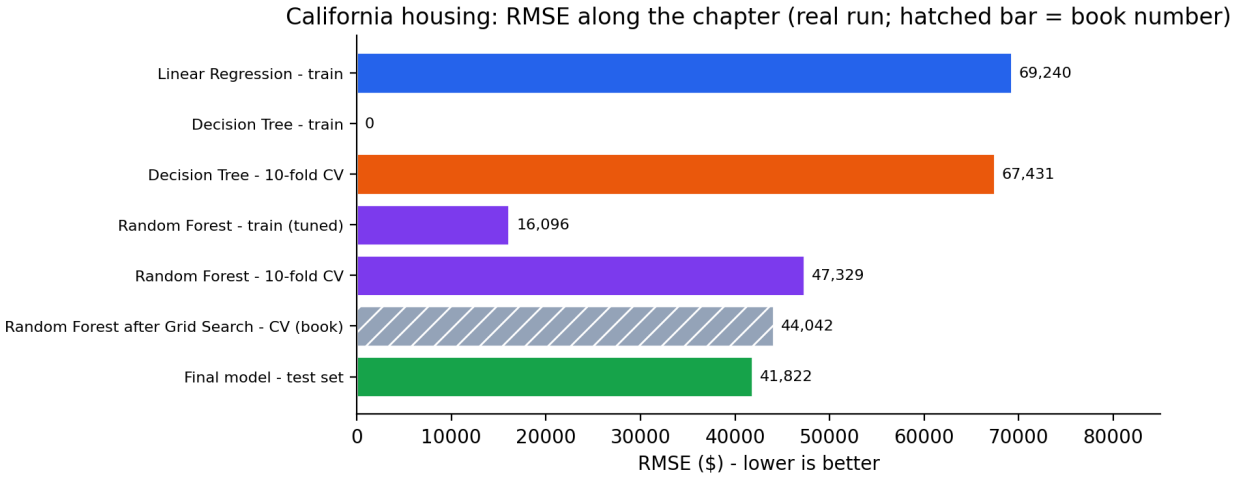
```
from sklearn.ensemble import RandomForestRegressor
forest_reg = make_pipeline(preprocessing, RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error", cv=10)
print(forest_rmses.mean(), forest_rmses.std())
```

Output (real run)

```
47328.8  2330.4
```

From your notebook

The concept of **Ensemble** (combining models): the book presents the idea of the Random Forest, a model composed of many models (most of the details come in Ch7). The results got better (for the book's example) and smaller; overfitting is still there but not with the same sharpness (less).



The journey of the numbers in the chapter (real run): notice that the RMSE on the train can be misleading (0 for the tree), so cross-validation is what reveals the truth.

15 Fine-tuning the model

① Grid Search

From your notebook

Let's assume you now have a short list of promising models (i.e. in the book's example the best was choosing the Random Forest model); now we need to tune them. So now we see how we tune them.

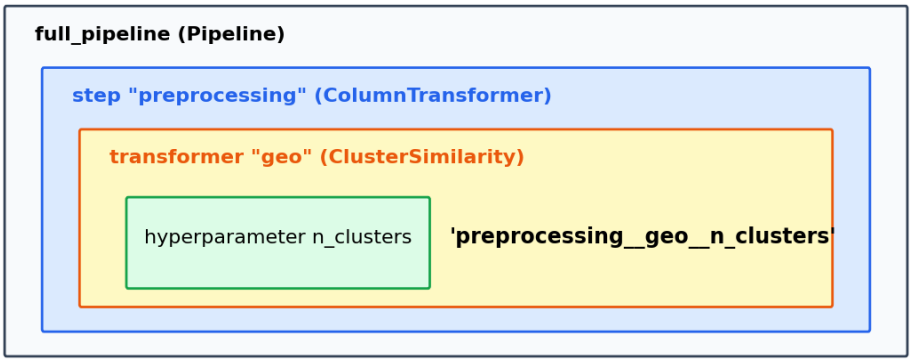
The problem: a model like Random Forest has `hyperparameters`, and their default values are not necessarily the best. And if you try all the values manually it will take a long time.

The solution: `GridSearchCV`

```
from sklearn.model_selection import GridSearchCV
full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])
param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10], 'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15], 'random_forest__max_features': [6, 8, 10]},
]
grid_search = GridSearchCV(full_pipeline, param_grid, cv=3, scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

Addition

From the book: the mean RMSE of the Random Forest in cross-validation ≈ 47,019 (std ≈ 1,034). Mine: **47,329** (± 2,330). And that is much better than the Tree. But the RMSE on the training set itself (the final model after tuning) = **16,096**, and it is much lower than the CV ← i.e. still overfitting. The solutions to overfitting according to the book: simplify the model, constrain it (regularization), or collect much more data.



The name of the hyperparameter = a path inside the nested structure, separated by `__`.

From your notebook

The idea: you give it a list of values for each hyperparameter and it tries all the possible combinations (all the possibilities), and evaluates each combination using Cross-Validation and tells you which combination is best.

The important part: the name `'preprocessinggeon_clusters'` is not a random name, it is a path to a hyperparameter deep inside a nested structure. Let's break it down:

- `preprocessing`: the step name in `full_pipeline` (which is the ColumnTransformer),
- `geo`: the transformer name inside the ColumnTransformer (which is `cluster_simil`),
- `n_clusters`: the hyperparameter inside the `geo` transformer.

sklearn separates the chain with a double underscore (`__`) and goes down one level inside the nested structure.

From your notebook

The arithmetic: the first dictionary: 3 × 3 = 9 combinations. The second: 2 × 3 = 6 combinations. The total is 15 combinations. And every combination is trained 3 times (`cv=3`), so the grand total = 15 × 3 = **45 training runs**. This shows why Grid Search takes a long time: every combination is repeated by the number of folds.

The result: `grid_search.best_params_` is the best combination, and the outputs. If the best value came out as the highest value of what you entered at the end of the range, then you must widen the range, because you are not sure this was the absolute best; you can try more and repeat.

Addition

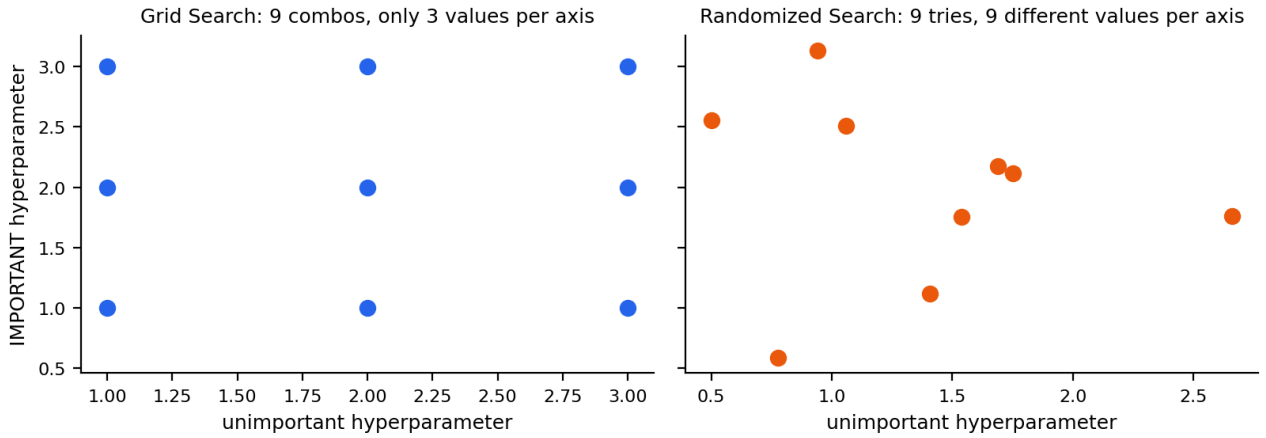
From the book: the best combination: `n_clusters=15` and `max_features=6`, and the best RMSE (cross-val) ≈ **44,042** (better than 47,019, which was with the default values). (The text in the book says `max_features=8` but the output and the table say 6.) Since 15 is the highest value tried for `n_clusters`, the book warns you to repeat the search with higher values.

`GridSearchCV` does `refit=True` by default: after it finds the best combination, it trains it again on the whole training set. And you can read all the results from `grid_search.cv_results_` (put them in a DataFrame).

(This table was not run here: 45 trainings of a Random Forest are heavy on a single-processor machine, so its numbers are from the book. But we took the best combination from it and actually ran it in section 16.)

② Randomized Search (a smarter alternative when there are many options)

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
param_distribs = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                  'random_forest__max_features': randint(low=2, high=20)}
rnd_search = RandomizedSearchCV(full_pipeline, param_distributions=param_distribs,
                                n_iter=10, cv=3, scoring='neg_root_mean_squared_error', random_state=42)
rnd_search.fit(housing, housing_labels)
```



The same number of attempts (9): Grid tries only 3 values for the important hyperparameter, and Random tries 9 different values.

From your notebook

The problem in Grid Search: if you have 6 hyperparameters and each one has 10 different values, you will need a million trainings (10^6), and this is not practical at all.

The essential difference: instead of trying all the combinations, it does a **fixed number of attempts** (`n_iter=10`), and in each attempt it picks a random value for each hyperparameter from `randint(low, high)` (a distribution between two values), a random sample of them, not all the combinations.

Why is it better?

1. **More flexibility:** it explores different values for each hyperparameter, with `n_iter=1000` , and not only the few values you wrote manually in Grid Search (i.e. 1000 different values for each hyperparameter).
2. **Saving time:** if a certain hyperparameter is not important (not influential), Grid Search multiplies the training time, but in Randomized Search the number of attempts is fixed.
3. **Full control of time:** instead of a "million forced trainings", you set the number of attempts (`n_iter`) according to the budget, and you control the size of the search space.

③ Halving Random Search / Halving Grid Search

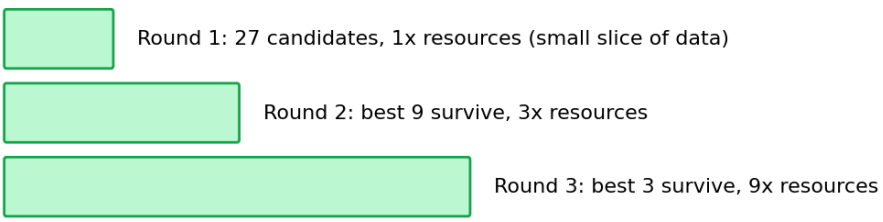
From your notebook

A quick remark: `HalvingRandomSearchCV` and `HalvingGridSearchCV` : additional options that are more efficient and faster. **The idea of Halving:** start with a lot of candidates using few resources, and complete with the best using more resources.

These search methods look for the best hyperparameters with a higher efficiency than ordinary search:

1. They start with a large number of candidates (hyperparameter combinations).
2. They try all the candidates with few resources (such as a small part of the training data).
3. They evaluate the results using Cross-Validation.
4. They keep only the best candidates.
5. In the next round they give the remaining candidates more resources.
6. They repeat the process until they reach the best candidates using full resources.

The goal: saving time and computational resources, or the ability to try a larger number of combinations in the same time.



Addition

The Halving classes are still "experimental" in sklearn, so you must enable them first with `from sklearn.experimental import enable_halving_search_cv` before you import them.

④ Ensemble Methods

From your notebook

Another way to improve the system's performance: try combining the best models with each other (like Random Forest + KNN, and take the average of their predictions). The idea of an ensemble, but this is the level of "combining completely different models", not "combining decision trees" like Random Forest (explained in Ch7).

⑤ Analyzing the best models (Feature Importances)

From your notebook

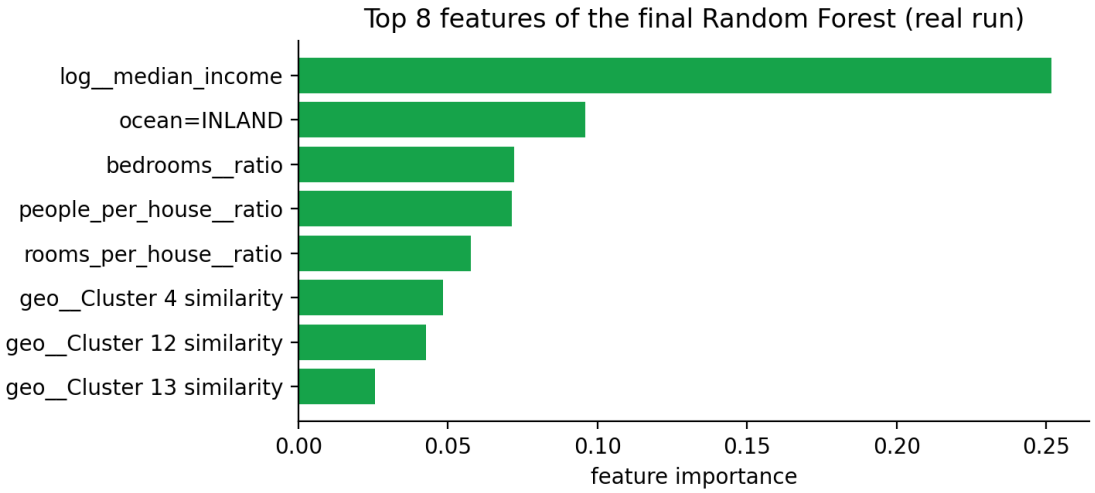
After `rnd_search` finishes, I have `best_params_` (the best hyperparameters) and `best_estimator_` (the best model). `final_model` is the full Pipeline (including the `"random_forest"` step), and `final_model["random_forest"]` gets the `RandomForestRegressor` inside the Pipeline.

`feature_importances_`: tells you how important each feature was in the Random Forest's decisions (the larger the number, the greater its weight in the model).

```
final_model = rnd_search.best_estimator_
feature_importances = final_model["random_forest"].feature_importances_
sorted(zip(feature_importances.round(3), final_model["preprocessing"].get_feature_names_out()),
       reverse=True)[:8]
```

Output (real run)

```
0.252 log__median_income
0.096 cat__ocean_proximity_INLAND
0.072 bedrooms__ratio
0.071 people_per_house__ratio
0.058 rooms_per_house__ratio
0.048 geo__Cluster 4 similarity
0.043 geo__Cluster 12 similarity
0.026 geo__Cluster 13 similarity
```



The most important features in the final model (real run): income (`median_income` / `log__median_income`) and location (`INLAND`) are at the top, and this is the same as what we expected from the correlation.

Addition

(The numbers above are from the model we trained with the best combination for the book.) The `zip` + `get_feature_names_out()` links each importance number to the feature name and `reverse=True` displays them from the largest to the smallest.

Remark: `feature_importances_` means the feature is "important" to the model at first glance, i.e. the Random Forest used it significantly in reducing the error while building the trees.

And a hint from the book: `SelectFromModel` trains a model (usually a Random Forest), looks at `feature_importances_`, and automatically removes the least useful features. And you also analyze the errors yourself: what cases the model errs in and why (adding a feature, cleaning outliers, etc.).

Note / Correction

An important warning: you must make sure the model works well on all the sub-categories (like the poor and rich areas), not only on the general average. If the model is weak on a certain category (like poor areas, for example) and is launched to Production and it gives bad results for a whole category of people, this can be an ethical risk. This is an important general principle in any real ML project.

16 Final evaluation and the Confidence Interval

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)

from scipy import stats
confidence = 0.95
```

```
squared_errors = (final_predictions - y_test) ** 2
np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
                         loc=squared_errors.mean(), scale=stats.sem(squared_errors)))
```

Output (real run)

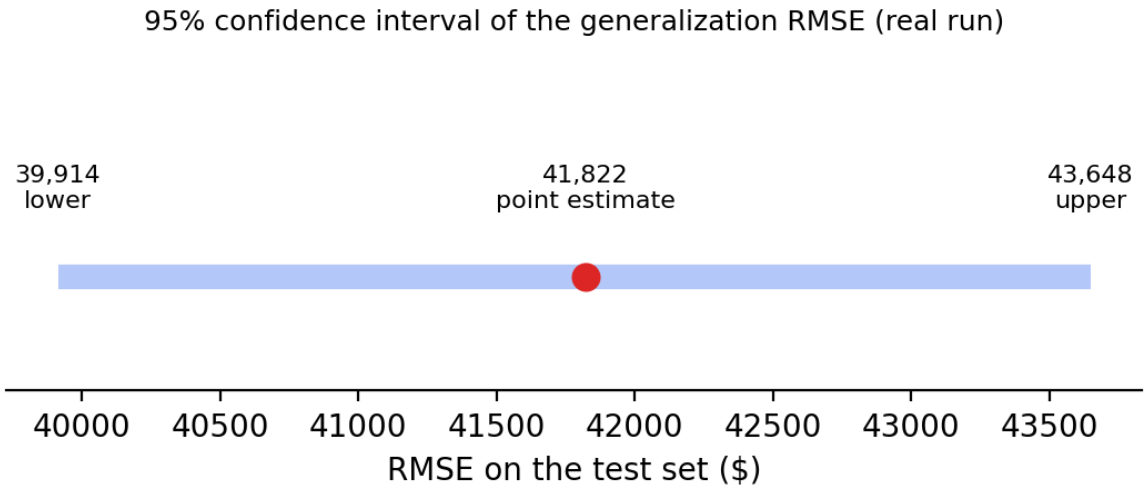
```
final RMSE = 41822.3
[39913.6 43647.5]      # 95% confidence interval
```

From your notebook

Here at last we touch the `test_set` and look at the outputs and so on. After Cross-Validation we will compute a **Confidence Interval** to measure the confidence range in the number, so we use statistical tools.

From your notebook

- `confidence = 0.95` : i.e. we compute a confidence range at the 95% level.
- `squared_errors` : here it computes the error of each prediction relative to the real value, squared.
- `len(squared_errors) - 1` = degrees of freedom = n − 1: if you have n errors, the degrees of freedom are n − 1. And this is required in computing the t-distribution (the function needs to know the number of degrees of freedom used in computing the Confidence Interval).
- `loc = squared_errors.mean()` : the center of the Confidence Interval, i.e. the mean of the squared errors (computed MSE).
- `scale = stats.sem(squared_errors)` : Standard error of the mean: the amount of uncertainty in estimating the mean of the squared errors.
- `np.sqrt` : converts the result from MSE to RMSE (the first number is the lower bound and the second is the upper bound).



Real run: RMSE on the test set = 41,822, and a 95% confidence interval from 39,914 to 43,648. (The book: 41,424 and an interval from 39,275 to 43,467.)

Addition

The RMSE on the test (41,822) is lower than that of the CV (44,042 in the book), and this is normal because each fold in CV trains on only part of the training set, whereas the final model was trained on all of it. This means the model generalizes well. And notice the difference between the RMSE on the train (16,096) and on the test: this gap is the overfitting we talked about.

Note / Correction

After you evaluate on the test set, do not try to "tune" the model so that the number improves; these improvements will not generalize to new data. That means the test set was used up and became like the validation.

17 Steps 7 and 8 (completed by me)

Addition

Present your solution: document what you learned: what worked and what failed, the assumptions and the limitations, and present the results clearly (graphics or easy sentences instead of just numbers).

Launch: prepare the code for production (cleaning, documentation, tests) and then deploy the model. The simplest way: save it and move it:

```
import joblib
joblib.dump(final_model, "my_california_housing_model.pkl")
final_model_reloaded = joblib.load("my_california_housing_model.pkl")   # custom classes
(ClusterSimilarity...) must be importable
```

Addition

A hint from the book: save every model you try (and the cross-validation scores and the predictions) so you can compare between them easily later.

Monitor & Maintain: the model loses its quality over time because the world changes (new data that differs from the training data), so you must monitor its performance continuously, retrain it with new data regularly, and automate this process (MLOps), and keep copies of the data and the models.

18 Quick review questions

Answer first in your head, then check the answers on the next page.

1. What is the difference between RMSE and MAE, and when do I prefer MAE?
2. If $R^2 = 0.8$, does that mean 80% of the predictions are correct?
3. What is the problem with random splitting when the data gets updated?
4. How does the hash (crc32) solve this problem?
5. Why is the row number (index) not a good ID?
6. What is the use of Stratified Sampling? And how did we apply it to median_income?
7. Why do we `fit` on the training set only, and `transform` the rest?
8. Why is OneHotEncoder better than OrdinalEncoder for categories with no order?
9. What is a Sparse matrix, and when do we need it?
10. What is the difference between Min-Max and Standardization? And which is better with an outlier?
11. What is the problem with a heavy-tail feature, and what is the solution?
12. When do I use Bucketizing with OneHot?
13. What is an RBF similarity feature?
14. Why did we use FunctionTransformer? And TransformedTargetRegressor?
15. What is the difference between Pipeline and ColumnTransformer?
16. Why is RMSE = 0 for the Decision Tree on the training set a bad sign? And how do we detect it?
17. Why does cross_val_score return negative values?
18. Grid Search: 15 combinations and cv=3, how many trainings? And why is Randomized better?
19. What does 'preprocessing*geo*n_clusters' mean?
20. What is the purpose of the Confidence Interval on the test set?

19 Review question answers

1. RMSE uses $\ell 2$ (squaring) so it is very affected by outliers, and MAE uses $\ell 1$ (absolute values) so it is calmer. If the outliers are many MAE is better; if they are rare RMSE is excellent.
2. No. It means the model explains about 80% of the variance in the target compared with predicting the mean only. Not Accuracy.
3. Each time the order changes, so rows may move from train to test, so the model has "seen" the test data (data leakage).
4. It converts a fixed ID into a fixed number, so the same row always goes to the same set (if hash < test_ratio × 2^32 it is test).
5. Because it changes if a row from the middle is deleted; the ID must be made of something fixed and unique.
6. It keeps the proportion of each category in train/test. We converted income into categories with pd.cut, then used stratify=income_cat, and afterwards deleted the column.
7. Because the statistics (median/mean) must be learned from the training only; and if they are computed from the test, leakage happens.
8. Ordinal gives numbers so the model assumes closeness/distance that is not real. One-hot makes a column for each category, so they are all equal and independent.
9. Storing only the positions of the 1s instead of all the zeros; it saves a lot of memory when the categories are many. toarray() or sparse_output=False if you want a normal array.
10. Min-Max puts the values in a fixed range so an outlier squeezes the rest. Standardization makes the mean 0 and the std 1 and is less affected by an outlier.
11. The model learns from it with difficulty. The solution: the logarithm (or the root / bucketizing) so the distribution gets closer to the bell shape.
12. When the feature is multimodal (several peaks), so each bucket is treated as a separate category.
13. A feature that measures closeness to a fixed value (for example age 35): $\exp(-\gamma(x-35)^2)$; it equals 1 at 35 and decreases as we move away.
14. The first converts any function into a transformer to be put in a pipeline; the second transforms the target and returns the predictions to their real value automatically.
15. Pipeline runs steps in order. ColumnTransformer applies a different pipeline to each group of columns and combines the output.
16. Because the model memorized the data: overfitting. We detect it with cross-validation (about 66-67 thousand in the example).
17. Because it expects a utility function (larger is better), so RMSE becomes neg_root_mean_squared_error and we flip it with a negative sign.
18. 45 trainings. Randomized tries a fixed number (n_iter) of random values, so it is more flexible and controls the time.
19. A path: the preprocessing step, then the geo transformer, then the hyperparameter named n_clusters, separated by __.
20. To know how accurate the estimate of the RMSE is (for example from X to Y with 95% confidence), not just one number.